

# AWS EKS DevOps Project – Kubernetes 2048 Game Deployment

This project demonstrates deployment of a containerized application on Amazon EKS using Kubernetes, AWS Load Balancer Controller, Ingress, IAM Roles for Service Accounts (IRSA), and Fargate. The project was built as part of hands-on DevOps practice for production-grade Kubernetes deployment.

## Project Architecture

- Amazon EKS Cluster
- Kubernetes Deployment & Service
- AWS Load Balancer Controller
- ALB Ingress Controller
- IAM Role & OIDC Integration
- AWS Fargate Profiles
- Metrics Server
- Public Access using ALB

## Technologies Used

- AWS EKS
- Kubernetes
- Docker
- kubectl
- eksctl
- AWS IAM
- Application Load Balancer (ALB)
- Git & GitHub

## Major Tasks Performed

1. Created EKS cluster using eksctl.
2. Configured kubectl access for the cluster.
3. Deployed the 2048 game application on Kubernetes.
4. Created Kubernetes Deployment and Service resources.
5. Installed Metrics Server in the cluster.
6. Configured AWS Load Balancer Controller.
7. Integrated IAM OIDC provider with EKS.
8. Created IAM Role for Service Account (IRSA).
9. Created ALB Ingress for external traffic routing.
10. Verified application access through AWS ALB DNS endpoint.

## Problems Faced & Troubleshooting

### 1. STS AssumeRoleWithWebIdentity – 403 AccessDenied

Issue: AWS Load Balancer Controller could not assume IAM role.

Fix: Correctly configured IAM OIDC provider and recreated IAM Service Account with proper role binding.

## **2. aws-load-balancer-controller Pending State**

Issue: Pods were stuck in Pending state.

Fix: Verified Fargate profile and Kubernetes scheduling configuration.

## **3. IAM Role Not Found**

Issue: AmazonEKSLoadBalancerControllerRole did not exist.

Fix: Created IAM role manually and attached required policies.

## **4. MalformedPolicyDocument Error**

Issue: Invalid JSON in trust relationship policy.

Fix: Corrected JSON syntax and updated OIDC ARN values properly.

## **5. Ingress ADDRESS Empty**

Issue: ALB was not being provisioned.

Fix: Fixed IAM permissions and restarted AWS Load Balancer Controller deployment.

# **Sample kubectl Commands**

```
kubectl get pods -A
kubectl get ingress -n game-2048
kubectl describe ingress ingress-2048 -n game-2048
kubectl get deploy -n kube-system
kubectl rollout restart deployment aws-load-balancer-controller -n kube-system
```

# **Project Outcome**

Successfully deployed a production-style Kubernetes application on AWS EKS using ALB Ingress. Implemented IAM-based authentication for Kubernetes service accounts and exposed the application publicly using AWS Load Balancer Controller.

# **Resume Points**

- Deployed containerized applications on Amazon EKS using Kubernetes.
- Configured AWS Load Balancer Controller with IAM Roles for Service Accounts (IRSA).
- Implemented Kubernetes Ingress with Application Load Balancer (ALB).
- Troubleshoot IAM, OIDC, and Kubernetes networking issues in EKS.
- Managed Kubernetes workloads on AWS Fargate.
- Automated cluster management using kubectl and eksctl.

*This project demonstrates hands-on DevOps and Kubernetes operational knowledge suitable for DevOps Engineer and Cloud Engineer roles.*